

RESQNET + Kavach

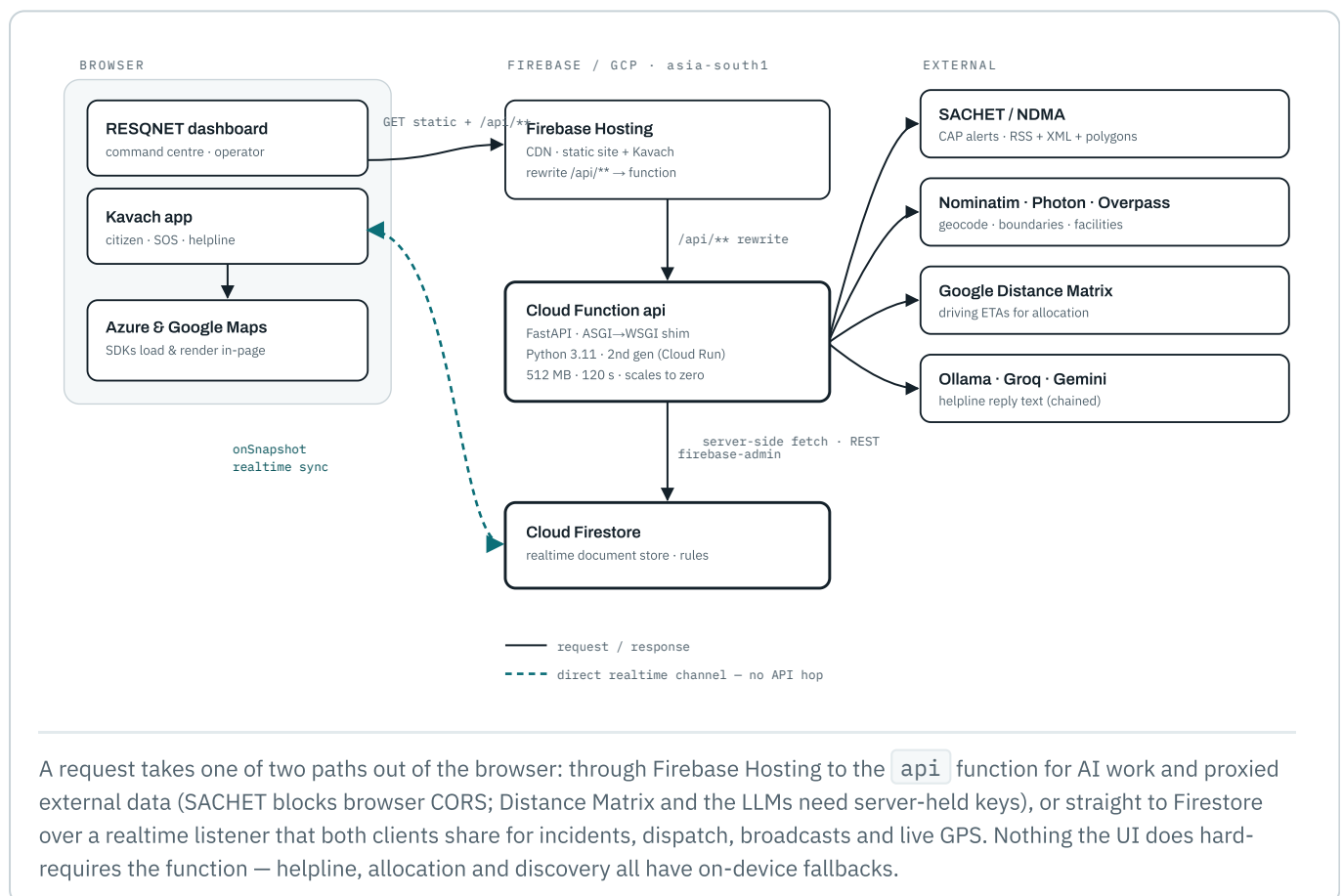
A disaster-response command centre (RESQNET) and a paired citizen app (Kavach), built on a static frontend, a FastAPI intelligence service, and Cloud Firestore as the realtime backbone — all served from one Firebase domain.

Project **sih-disaster-management-6b931** (Blaze) Region **asia-south1**
 Live **sih-disaster-management-6b931.web.app** API **/api/**** → Cloud Function

01 ARCHITECTURE

Two paths out of the browser

The dashboard and the citizen app are plain static bundles. Everything dynamic goes one of two ways: through **Firebase Hosting** to the `api` Cloud Function for AI work and any external data that browsers cannot fetch directly, or straight to **Cloud Firestore** over a realtime listener that both clients share. Map SDKs load into the page and talk to Azure / Google directly.



02 FRONTEND

Static bundles, no build step

The RESQNET dashboard and the Kavach citizen app are hand-written ES-module JavaScript served as flat files — no bundler, no framework runtime.

CONCERN	CHOICE	NOTES
Language	Vanilla JS (ES modules)	No React/Vue; DOM APIs directly. <code>script.js</code> , <code>district-gate.js</code> , <code>relocation-sim.js</code>
Styling	<code>cdn.tailwindcss.com</code>	Tailwind play CDN with forms + container-queries plugins; plus <code>style.css</code>
Icons	<code>font-awesome 6.4.0</code>	via cdnjs
Operator map	Azure Maps Web SDK	<code>atlas mapcontrol v3</code>
Relocation map	Google Maps JS API	Places API (New) + geometry; region=IN for official boundaries
Realtime / DB	<code>firebase-js-sdk 10.7.1</code>	modular build from gstatic; Firestore + App + Analytics
Voice	Web Speech API	<code>webkitSpeechRecognition</code> (STT) + <code>speechSynthesis</code> (TTS) for the Kavach helpline / IVR
Device	Notification, Geolocation	push alerts; <code>watchPosition</code> streams live caller GPS during an SOS
Same-browser bus	<code>BroadcastChannel</code> + <code>localStorage</code>	demo fallback when Firestore rules are not deployed
i18n	Hand-rolled EN / हिंदी / Hinglish	phrase maps + text-node sweep + <code>MutationObserver</code>
Bundled data	<code>india-districts.js</code>	36 states / 725 districts — instant district picker, no API

03 BACKEND — SIH CITIZEN INTELLIGENCE API

One FastAPI app, deployed as a function

`SIH_HAZARD_MANAGEMENT_ai/` — a single FastAPI application mounted under `/api`, wrapped as a 2nd-gen Cloud Function so `firebase deploy` ships it alongside the site.

CONCERN	CHOICE	NOTES
Runtime	Python 3.11	Cloud Functions 2nd gen (Cloud Run under the hood)
Web framework	<code>fastapi</code> + <code>uvicorn</code>	ASGI; thin root app mounts the real API at <code>/api</code> , keeps <code>/health</code>
Schemas	<code>pydantic v2</code>	<code>app/schemas.py</code> — request/response + domain models
Function glue	<code>firebase-functions</code>	<code>main.py</code> + a hand-rolled inline ASGI→WSGI shim (a2wsgi deadlocked on Cloud Run gen2)
Firestore	<code>firebase-admin</code>	Admin SDK; credentials via env JSON, key file, or ADC (function runtime SA)
HTTP client	<code>requests</code>	SACHET proxy, LLM REST calls, Distance Matrix
Uploads	<code>python-multipart</code>	<code>/voice-analyze</code> audio

CONCERN	CHOICE	NOTES
Transcription	faster-whisper	optional lazy import; 501 if absent (browser STT covers the helpline)
HTTP surface (all under /api).		
ENDPOINT	PURPOSE	
POST /analyze	Extract facts + severity + priority from one citizen message	
POST /incidents	Analyse & persist an incident to Firestore (+ /aggregation, /{id}, PATCH)	
GET /resources	Facility inventory (+ POST /resources/bulk upsert from district discovery)	
GET /allocate	Advisory resource→cluster plan (+ POST /allocate/commit to apply)	
POST /helpline/turn	One conversational triage turn; files a geo-tagged incident when done	
GET /sachet/{path}	Server-side NDMA fetch — browser-CORS bypass, path allowlist, 120 s cache	
POST /voice-analyze	Audio → transcript → the same NLP pipeline	

Deterministic core, LLM only for words

Severity and priority are computed by a fixed pipeline so results are explainable and reproducible. A language model is used *only* to phrase the helpline’s spoken reply — never to decide urgency.

STAGE	APPROACH	NOTES
Fact extraction	Rules + optional LLM	analyzer.py — regex/keyword extraction of needs, headcount, trapped, injured, vulnerable people, hazards; can defer to Ollama qwen3:4b
Severity	Deterministic features	severity.py — immediate-rescue / medical / multiple-people / vulnerable flags
Priority	Deterministic score + level	priority.py — CRITICAL / HIGH / MEDIUM / LOW with reasons
Urgency model	Pure-Python linear model	LinearUrgencyModel + sigmoid; weights in urgency_weights.json, trained by gradient descent on synthetic scenarios + expert labels (no numpy/sklearn)
Aggregation	Spatial clustering	aggregation.py — groups incidents into area-level demand
Allocation	Severity-tiered greedy match	resource_allocator.py — ranks by distance/ETA, availability and a spread factor; advisory then committable
Helpline reply	LLM chain → script	Ollama qwen3:4b → Groq → Gemini (gemini-2.0-flash) → fixed EN/HI/Hinglish script

Free and official first, Google when billed

SOURCE	USED FOR	ACCESS
SACHET / NDMA	Official CAP disaster alerts + polygons	via /api/sachet
Nominatim (OSM)	Geocoding + district boundary polygons	browser, direct
Photon (komoot)	Hospital / police / fire discovery	browser, CORS-ok
Overpass API	Facility discovery fallback	browser, direct
Google Places (New)	Facility discovery when Maps billing is on	browser, keyed
Google Distance Matrix	Driving ETAs for allocation	backend, server key
Azure Maps	Operator base map + Kavach weather	browser, sub key

Graceful degradation is designed in. No SACHET proxy → alerts panel is blank but everything else runs. No Distance Matrix → ETAs fall back to haversine ×1.3. No LLM key → the helpline uses its fixed script. Backend unreachable → Kavach writes incidents straight to Firestore and the dashboard runs an on-page allocator.

One Firebase project, one domain

Firebase project `sih-disaster-management-6b931` on the Blaze plan. A Firebase project is a GCP project, so the function, database and hosting share it.

SERVICE	ROLE	CONFIG
Firebase Hosting	Static site + CDN + API front door	<code>rewrite /api/** → fn api</code>
Cloud Functions (2nd gen)	The FastAPI backend	<code>python311 · 512Mi · 120s</code>
Cloud Run + Build + Artifact Registry	Build & run substrate for the function	<code>by firebase deploy</code>
Cloud Firestore	Realtime store + security rules	<code>firestore.rules</code>
Application Default Credentials	Function → Firestore auth	<code>...-compute@developer</code>
Firebase Analytics	Optional page analytics	<code>best-effort</code>

Alternatives are kept in the repo but unused: direct Cloud Run (`Dockerfile`), Render (`render.yaml`), a Cloudflare Worker for SACHET (`sachet-worker/`), and a local Node SACHET proxy (`sachet-proxy.js`).

07 FIRESTORE DATA MODEL

Six collections, one shared bus

COLLECTION	WRITTEN BY	CONTENTS
<code>incidents</code>	Kavach + backend + dashboard	Citizen SOS / helpline reports; status UNASSIGNED → ASSIGNED → IN_PROGRESS → RESOLVED
<code>resources</code>	Backend (Admin SDK)	Discovered facilities + unit availability
<code>relocations</code>	Dashboard	AI dispatch decisions, animated on the map, streamed to Kavach
<code>caller_locations</code>	Kavach	Live citizen GPS, keyed by incident id
<code>broadcasts</code>	Dashboard	Authority SACHET-style broadcasts to every Kavach device in the area
<code>safe_checkins</code>	Kavach	“I’m safe” check-ins during an event

Rules are prototype-open (read-all, create/update true, no deletes). The AI → citizen link is entirely Firestore: `firebase-bridge.js` writes, `kavach-firebase.js` subscribes with `onSnapshot`.

08 DEV & DEPLOY

Two commands

- **Backend** — `cd SIH_HAZARD_MANAGEMENT_ai && firebase deploy --only functions` (needs local Python 3.11 for the discovery venv; `py install 3.11`).
- **Frontend + rules** — `cd SIH_HAZARD_MANAGEMENT && firebase deploy --only hosting,firestore:rules`.
- **Config** — `config.js` is the single edit point: Firebase web config, `RESQNET_AI_API_BASE=/api`, `RESQNET_SACHET_PROXY=/api/sachet`, map keys.

- **Logs** — `firebase functions:log --only api`.
- **Toolchain** — Firebase CLI, the `py` install manager, Git. `gcloud` / Wrangler / Docker only for the alternative hosting paths.

09 REPOSITORY LAYOUT

PATH	WHAT'S THERE
SIH_HAZARD_MANAGEMENT/	Dashboard (index.html, script.js, district-gate.js, relocation-sim.js, config.js, firebase-bridge.js), firebase.json, firestore.rules
└─ Kavach/	Citizen app (kavach.html, app.js, kavach-firebase.js)
└─ sachet-worker/	Cloudflare Worker port of the SACHET proxy (alternative)
SIH_HAZARD_MANAGEMENT_ai/	app/ FastAPI modules, main.py function entry, firebase.json / .firebaserc, scripts/, tests/, Dockerfile / render.yaml